CALCUL LAMBDA

Reduceți expresia lambda $E = (\lambda x.(x\ \ (\lambda y.z\ \ x))\ \ \lambda x.x)$
Soluție:
$\rightarrow (\lambda x.(x\ \ z)\ \ \lambda x.x) \rightarrow (\lambda x.x\ \ z) \rightarrow z$

1) Reducere beta pentru $(\lambda y.z\ x)$ -> z
2) Reducere beta pentru $(\lambda x.(x\ z)\ \lambda x.x)$ -> $(\lambda x.x\ z)$
3) Reducere beta pentru $(\lambda x.x\ z)$ -> z

1. Reduceți expresia lambda $E = (y\ \ (\lambda x.\lambda x.x\ \ (\lambda y.y\ \ y)))$
   Soluție:
   $\rightarrow (y\ \ (\lambda x.\lambda x.x\ \ y)) \rightarrow (y\ \ \lambda x.x)$

1) Reducere beta pentru (λy.y y) -> y
2) Reducere beta pentru (λx.λx.x y) -> λx.x (nu există apariții libere ale variabilei x)

1. Reduceți la forma normală următoarea expresie, ilustrând pașii de reducere:
   $\lambda x.\lambda y.((\lambda x.\lambda y.x\ \ (y\ \ x))\ \ (x\ \ y))$
   Soluție:
   $\lambda x.\lambda y.((\lambda x.\lambda y.x\ \ (y\ \ x))\ \ (x\ \ y)) \rightarrow_\alpha \lambda x.\lambda y.((\lambda x.\lambda z.x\ \ (y\ \ x))\ \ (x\ \ y)) \rightarrow_\beta$
   $\lambda x.\lambda y.(\lambda z.(y\ \ x)\ \ (x\ \ y)) \rightarrow_\beta \lambda x.\lambda y.(y\ \ x)$

1. Reduceți la forma normală următoarea expresie, ilustrând pașii de reducere:
   $\lambda x.\lambda y.((\lambda x.\lambda y.y\ \ (x\ \ y))\ \ (y\ \ x))$
   Soluție:
   $\lambda x.\lambda y.((\lambda x.\lambda y.y\ \ (x\ \ y))\ \ (y\ \ x)) \rightarrow_\beta \lambda x.\lambda y.(\lambda y.y\ \ (y\ \ x)) \rightarrow_\beta \lambda x.\lambda y.(y\ \ x)$

Timp de lucru 2 ore. 100p necesare pentru nota maxima

1. Determinați forma normală pentru următoarea expresie, ilustrând pașii de reducere:
   $((\lambda x.\lambda y.\lambda z.(x\ \ y)\ \ \lambda x.x)\ \ z)$
   Soluție:
   $((\lambda \underline{x}.\lambda y.\lambda z.(\underline{x}\ \ y)\ \ \lambda x.x)\ \ z) \rightarrow_\beta (\lambda \underline{y}.\lambda z.(\lambda x.x\ \ \underline{y})\ \ z) \rightarrow_\alpha (\lambda \underline{y}.\lambda w.(\lambda x.x\ \ \underline{y})\ \ z) \rightarrow_\beta \lambda w.(\lambda \underline{x}.\underline{x}\ \ z) \rightarrow_\beta$
   $\lambda w.z$

1. Determinați forma normală pentru următoarea expresie, ilustrând pașii de reducere:
   $((\lambda x.\lambda y.\lambda z.(y\ \ x)\ \ y)\ \ \lambda z.z)$
   Soluție:
   $((\lambda \underline{x}.\lambda y.\lambda z.(y\ \ \underline{x})\ \ y)\ \ \lambda z.z) \rightarrow_\alpha ((\lambda \underline{x}.\lambda w.\lambda z.(w\ \ \underline{x})\ \ y)\ \ \lambda z.z) \rightarrow_\beta (\lambda \underline{w}.\lambda z.(\underline{w}\ \ y)\ \ \lambda z.z) \rightarrow_\beta$
   $\lambda z.(\lambda \underline{z}.\underline{z}\ \ y) \rightarrow_\beta \lambda z.y$

1. Determinați forma normală pentru următoarea expresie, ilustrând pașii de reducere:
   $((\lambda x.\lambda y.\lambda z.(x\ \ y)\ \ \lambda x.y)\ \ a)$
   Soluție:
   $((\lambda \underline{x}.\lambda y.\lambda z.(\underline{x}\ \ y)\ \ \lambda x.y)\ \ a) \rightarrow_\alpha ((\lambda \underline{x}.\lambda w.\lambda z.(\underline{x}\ \ w)\ \ \lambda x.y)\ \ a) \rightarrow_\beta (\lambda \underline{w}.\lambda z.(\lambda x.y\ \ \underline{w})\ \ a) \rightarrow_\beta$
   $\lambda z.(\lambda x.y\ \ a) \rightarrow_\beta \lambda z.y$

RACKET - RECURSIVITATE PE STIVĂ VS COADĂ

2.
a)
Pentru funcția f de mai jos:

```
(define (f L)
  (if (or (null? L) (null? (cdr L)))
      L
      (cons (max (car L) (cadr L))
            (f (cddr L)))))
```

i) Argumentați ce tip de recursivitate (stivă/coadă) folosește f. (1p tipul, 2p argumentația)
ii) Spuneți în cuvinte ce calculează f. (2p)
iii) Definiți funcția ff, care are același efect cu f, dar folosește celălalt tip de recursivitate. (5p)

Soluție 2a:
i) stivă - apelul recursiv nu e în poziție finală.
ii) se parcurg elementele 2 câte 2 și se reține maximul din fiecare pereche; pentru o listă cu număr impar de elemente, ultimul element este copiat ca atare în rezultat.
iii)

```
; nu se acordă puncte dacă recursivitatea nu este pe coadă
(define (ff L)
  (let loop ((L L) (acc '())) ; 1p acc
    (if (or (null? L) (null? (cdr L)))
        (reverse (append L acc))      ; 1p ordinea corectă
                                      ; 1p returul acc pe cazul de bază
        (loop (cddr L) (cons (max (car L) (cadr L)) acc)))))
                              ; 2p apelul recursiv cu actualizarea acc
```

.
Pentru funcția f de mai jos:

```
(define (f L)
  (cond ((or (null? L) (null? (cdr L))) L)
        ((equal? (car L) (cadr L)) (f (cdr L)))
        (else (cons (car L) (f (cdr L))))))
```

i) Argumentați ce tip de recursivitate (stivă/coadă) folosește f. (1p tipul, 2p argumentația)
ii) Spuneți în cuvinte ce calculează f. (2p)
iii) Definiți funcția ff, care are același efect cu f, dar folosește celălalt tip de recursivitate. (5p)

```
Soluție 2b:
i)   stivă - al doilea apel recursiv nu e în poziție finală.
ii)  se elimină duplicatele consecutive din listă.
iii)

; nu se acordă puncte dacă recursivitatea nu este pe coadă
(define (ff L)
  (let loop ((L L) (acc '())) ; 1p acc
    (cond ((or (null? L) (null? (cdr L))) (reverse (append L acc)))
                                ; 1p ordinea corectă
                                ; 1p returul acc pe cazul de bază
          ((equal? (car L) (cadr L)) (loop (cdr L) acc))
                                ; 1p apelul cu acc nemodificat
          (else (loop (cdr L) (cons (car L) acc))))))
                                ; 1p apelul cu acc actualizat
```

2. Ce tip de recursivitate are implementarea de mai jos?

```
(define (z x y)
    (if (or (null? x) (null? y))
        '()
        (cons (car x) (cons (car y) (z (cdr y) (cdr x))))))
```

Definiți (tot în Racket) cât mai eficient funcția `zz` care are același efect cu `z`, dar folosește un alt tip de recursivitate.

Justificați pe scurt eficiența soluției voastre.

Soluție:

Recursivitate pe stivă – o vom transforma în recursivitate pe coadă.

```
(define (zz x y)
    (let z ((x x) (y y) (r '()))
        (if (or (null? x) (null? y))
            (reverse r)
            (z (cdr y) (cdr x) (cons (car y) (cons (car x) r))))))
```

Eficiență:

- recursivitatea pe coadă elimină nevoia de a ocupa spațiu pe stiva de apeluri recursive;
- adăugarea cu `cons` în acumulatorul `r` (și inversarea lui `r` la final) duce la o construcție în timp liniar, spre deosebire de timp pătratic dacă făceam `append` la finalul lui `r`.

FUNCȚIONALE - RACKET

Definiți, în Racket, funcția odd-pos care selectează doar elementele aflate pe poziții impare într-o listă L.
Lista este indexată de la 0.
Vă poate fi de folos (dar nu este neapărat necesară) funcția range (ex: (range 5) => '(0 1 2 3 4) ; (range 8) => '(0 1 2 3 4 5 6 7)).

ex:
(odd-pos '(5 6 5 4 3 4 5)) => '(6 4 4)

```
; cu map + filter
(define (odd-pos L)
  (let ((p (map cons L (range (length L)))))
; 4p împerechere element-poziție (2p folosire map, nu un zip inventat)
    (map car ; 2p extragerea elementelor listei din perechi
         (filter (λ(x) (odd? (cdr x))) p))))
             ; 4p filtrarea pozițiilor impare (2p predicat corect)
```

Definiți, în Racket, funcția alternate care primește o listă L și 2 funcții f1 și f2 și aplică alternativ f1 și f2 pe elementele lui L.
ex:
(alternate '(1 2 3 4 5 6 7) add1 sub1) => '(2 1 4 3 6 5 8)

```
(define (alternate L f1 f2)
(let ((p (map cons L (range (length L)))))
  (map (lambda (x) (if(odd? (cdr x)) (f2 (car x))  (f1 (car x)))) p)))
```

```
; cu map
(define (alternate L f1 f2)
  (let* ((l  (length L))
         (fL (flatten (make-list l (list f1 f2)))))
; 4p crearea listei de funcții (2p aplatizare cu flatten sau apply)
    (map (λ (f x) (f x)) (take fL l) L)))
; 4p aplicarea listei de funcții pe L
; 2p lungimea corectă a listei de funcții
```

Definiți, în Racket, funcția compare-seq care primește o listă numerică L și întoarce o listă de elemente de tip 'LT, 'EQ sau 'GT după relația de ordine între elementele consecutive ale listei L ('LT atunci când primul e mai mic decât al doilea, 'EQ când sunt egale, 'GT când primul e mai mare).

Soluție 3c:

```
(define (compare-seq L)
  (let* ((l  (length L))
         (L1 (take L (- l 1)))
                ; 2p crearea listei fără ultimul element
         (L2 (cdr L)))
                ; 2p crearea listei fără primul element
                ; 4p aplicarea funcției binare pe cele 2 liste
                ; 2p funcția binară a lui map
    (map (λ (x y) (cond ((= x y) 'EQ)
                        ((< x y) 'LT)
                        ((> x y) 'GT))) L1 L2)))
```

```
(define (compare-seq L)
  (
   let ((p (map cons (take L (sub1 (length L))) (cdr L) )))
    (map (lambda (x)
           (if(equal? (cdr x) '())
              '()
              (if(= (car x) (cdr x))
                 'EQ
                 (
                  if(> (car x) (cdr x))
                    'GT
                    'LT
                  )
                 )
              )
           ) p)
   )
  )
```

Definiți, în Racket, funcția eq-sums? care primește o listă numerică L cu număr par de elemente și întoarce true dacă toate sumele provenite din adunarea elementelor de pe poziții simetrice față de mijlocul listei sunt egale, respectiv false în caz contrar.

```
(define (eq-sums? L)
  ( let* ((R (reverse L))
          (sum-list (map + L R))
          (first-elem (car sum-list))
          (verif-list (filter (lambda (x) (= x first-elem)) sum-list)))
     (if(= (length sum-list) (length verif-list))
     #t
     #f)
   )
  )
```

```
(define (eq-sums? L)
  (or (null? L) ; 0p cazul de listă vidă
      (let ((R (reverse L))) ; 2p inversarea listei
        (null? (filter (λ (x) (not (equal? x (+ (car L) (car R)))))
                ; 2p predicat corect filter
                ; 2p prelucrare filter (aici null?)
                       (map + L R))))))
               ; 4p adunarea celor 2 liste
```

Se dă o listă L care conține numere și/sau liste de numere (maxim un nivel de imbricare). Să se definească, în Racket, funcția **two-sums** care întoarce o pereche de valori, reprezentând suma numerelor la "adâncime 0" din L, respectiv suma numerelor la "adâncime 1" din L.

ex: `(two-sums '(1 2 (3 4) 5 (6) 7))` va întoarce `'(15 . 13)`, pentru că $1+2+5+7=15$, iar $3+4+6=13$

Soluție:

```
(define (two-sums L)
    (let ((outside (filter (compose not list?) L))
          (inside (apply append (filter list? L))))
        (cons (apply + outside) (apply + inside))))
```

MORE RACKET

2. Se dă următorul cod Racket:

```
(define computation (delay (+ 5 5)))
(* 5 5)
(define (f x) (cons x (force computation)))
(map f '(1 2 3 4))
```

(a) De câte ori se realizează adunarea?
(b) Prima evaluare a adunării se realizează înainte sau după înmulțire?
(c) Rescrieți codul pentru `computation` și pentru `f` folosind închideri funcționale în loc de promisiuni și răspundeți din nou la întrebările de la (a) și (b).

Soluție:
(a) o singură dată, la prima evaluare a lui `computation`.
(b) după înmulțire, atunci când se apelează prima oară `(force computation)`
(c)

```
(define computation (λ () (+ 5 5)))
(* 5 5)
(define (f x) (cons x (computation)))
(map f '(1 2 3 4))
```

acum se apelează de 4 ori, la fiecare evaluare a lui `computation`; dar prima dată tot după înmulțire.

3. Date fiind două liste de numere `L1` și `L2`, scrieți în Racket codul care produce o listă de perechi `(x . n)`, unde `x` este un element din `L1`, iar `n` este numărul de apariții ale lui `x` în `L2`. E.g. pentru `L1 = (1 4 5 3)` și `L2 = (1 3 2 4 1 5 3 9)` rezultatul este `((1 . 2) (4 . 1) (5 . 1) (3 . 2))`. Nu folosiți recursivitate explicită.

Soluție:

```
(map (lambda (x) (cons x (length (filter ((curry equal?) x) L)))) '(1 4 5 3))
```

sau

```
(map (lambda (x) (cons x (length (filter (lambda (y) (equal? x y)) L)))) '(1 4 5 3))
```

```
(define (f L1 L2)
  (map (lambda (x) (
                    let ((num (length (filter (lambda (y) (equal? x y)) L2))))
                      (cons x num)
                    )) L1)
  )
```

FLUXURI RACKET

Pornind de la fluxul numerelor naturale din Racket, definiți fluxul fluxurilor de numere naturale consecutive, din 2 în 2, din 3 în 3 ș.a.m.d. Exemplu: `((0 1 2 3 ...) (0 2 4 6 ...) (0 3 6 9 ...) ...)`. Considerați că `naturals` este deja definit și că puteți utiliza orice funcționale pe fluxuri, discutate la curs și laborator.

Hint: În loc să construiți de la zero fiecare flux interior, gândiți-vă cum îl puteți obține pe baza fluxului anterior.

Rezolvare:

```
(define multiples
  (stream-cons naturals
               (stream-map (λ (s) (stream-zip-with + s naturals))
                           multiples)))
```

Definiți în Racket următorul flux infinit: ((1) (1 2 1) (1 2 3 2 1) (1 2 3 4 3 2 1) ...).

Hint: În loc să construiți de la zero fiecare listă interioară, gândiți-vă cum o puteți obține pe baza listei anterioare.

Rezolvare:

```
(define pyramids
  (stream-cons '(1)
               (stream-map (λ (L)
                             (append '(1) (map add1 L) '(1)))
                           pyramids)))
```

Pornind de la un flux infinit de liste, construiți fluxul reuniunilor parțiale ale acestuia, în care lista de pe poziția i conține reuniunea primelor i liste din fluxul inițial, iar prima listă este vidă. Atenție! Reuniunile NU trebuie să conțină duplicate! Exemplu: pentru fluxul ((2 3) (3 4 5) (2 4) (1) ...) se obține fluxul (() (2 3) (2 3 4 5) (2 3 4 5) (2 3 4 5 1) ...).

```
(define (partial-unions sets)
  (letrec
      ([out
        (stream-cons
         '()
         (stream-zip-with (λ (union set)
                            (append union
                                    (filter (λ (x)
                                              (not (member x union)))
                                            set)))
                          out
                          sets))])
    out))
```

EVALUAREA UNEI EXPRESII ÎN HASKELL

Fie programul Haskell de mai jos:

```
lst = [(1+2, 3+4), (3+4, 5+6), (5+6, 7+8)]

find ((7,x) : _) = x
find (_ : pairs) = find pairs
```

Descrieți cum decurge, pas cu pas, evaluarea expresiei (find lst).

```
find lst
find ((1+2, 3+4) : ...)
find ((3, 3+4) : ...)
find ((3+4, 5+6) : ...)
find ((7, 5+6) : ...)
5+6
11
```

```
head $ ([1] ++ [2]) ++ [3]
```

```
head $ ([1] ++ [2]) ++ [3]       -- (1)
head $ (1 : ([] ++ [2])) ++ [3]  -- (2)
head $ 1 : (([] ++ [2]) ++ [3])  -- (3)
1                                -- (4)
```

SINTEZĂ TIP HASKELL

Sintetizați, pas cu pas, tipul următoarei funcții în Haskell:

```
f x y = fst x : map (snd x) y
```

```
f :: a -> b -> c
x :: a
y :: b
fst :: (d, e) -> d
snd :: (d, e) -> e
a = (d, e)
fst x :: d
snd x :: e
map :: (g -> h) -> [g] -> [h]
e = g -> h
b = [g]
c = [h]
d = h
f :: (h, g -> h) -> [g] -> [h]

Notă: Am folosit aceleași variable de tip la fst și snd pentru
concizie, întrucât se aplică asupra aceluiași parametru, x. În mod
normal, se utilizează alte variabile de tip, e.g. snd :: (g, h) -> h.
```

Sintetizați, pas cu pas, tipul următoarei funcții în Haskell:

`f x y = head x y`

Atenție! Simpla mențiune a tipului final, fără pași intermediari, nu va fi punctată!

```
f :: a -> b -> c
x :: a
y :: b
head :: [d] -> d
a = [d]
head x = d
d = e -> g
e = b
c = g
f :: [b -> g] -> b -> g
```

Sintetizați, pas cu pas, tipul următoarei funcții în Haskell:

`f x y = [tail x ++ fst y, snd y]`

Atenție! Simpla mențiune a tipului final, fără pași intermediari, nu va fi punctată!

```
f :: a -> b -> c
x :: a
y :: b
tail :: [d] -> [d]
a = [d]
tail x :: [d]
fst :: (e, g) -> e
snd :: (e, g) -> g
b = (e, g)
fst y :: e
snd y :: g
e = [d]
g = [d]  (din aceeași listă cu elemente de tipul e)
f :: [d] -> ([d], [d]) -> [[d]]
```

Sintetizați, pas cu pas, tipul următoarei funcții în Haskell:

```
f x y = (f x, f y)
```

```
f :: a -> b -> c
x :: a
y :: b
a = b
f x :: a -> c
```

```
f y :: a -> c
c = (a -> c, a -> c)
Eroare de sinteză, întrucât expresia de tip din dreapta conține
strict variabila c.
```

5. Sintetizați în Haskell, pas cu pas, tipul funcției `g`:

   ```
   g fs xs = zipWith (\f x -> f x) fs xs
   ```

   cunoscând tipul funcției `zipWith`:

   ```
   zipWith :: (a -> b -> c) -> [a] -> [b] -> [c]
   ```

   Soluție:

   ```
   f :: a = d -> e
   x :: b = d
   c = e
   fs :: [a] = [d -> e]
   xs :: [b] = [d]
   g :: [d -> e] -> [d] -> [e]
   ```

4. Sintetizați tipul următoarei funcții în Haskell: `f x y = x y (y x)`
   Soluție:
   `x :: a -> b -> c`
   `y :: a`, dar și `y :: (a -> b -> c) -> b` $\Rightarrow$ `a = (a -> b -> c) -> b` Eroare, deoarece `a` nu poate unifica cu o expresie de tip care îl conține strict pe `a`.

SUPRAÎNCĂRCARE FUNCȚII HASKELL

Supraîncărcați în Haskell reprezentarea sub formă de șir de caractere pentru funcții unare, cu parametru numeric, astfel încât o astfel de funcție să fie vizualizată printr-o listă de perechi, care conțin pe prima poziție numerele de la 0 la 5, iar pe a doua, valorile funcției în aceste puncte. De exemplu, show (*2) = "[(0,0),(1,2),(2,4),(3,6),(4,8),(5,10)]".

```
instance (Num a, Show a, Enum a, Show b) => Show (a -> b) where
    show f = show $ map (\x -> (x, f x)) [0..5]
```

Supraîncărcați în Haskell operatorul de ordonare a funcțiilor unare, cu parametru numeric, astfel încât funcția f este mai mică sau egală cu funcția g dacă valorile lui f în punctele 0..100 sunt mai mici sau egale cu valorile lui g în aceleași puncte. Comparațiile se fac pentru aceiași parametri. De exemplu, (+1) <= (+2), întrucât (+1) 0 <= (+2) 0, (+1) 1 <= (+2) 1 etc.

```
instance (Num a, Enum a, Ord b) => Ord (a -> b) where
    f <= g = and $ zipWith (<=) (map f [0..100]) (map g [0..100])
```

Scrieți în Haskell o posibilă instanță a clasei de mai jos, conținând o implementare neconstantă a funcției convert.

```
class Convertor c where
    convert :: Num a => [a] -> c a
```

```
instance Convertor Maybe where
    convert [] = Nothing
    convert xs = Just $ sum xs
```

Scrieți în Haskell o posibilă instanță a clasei de mai jos, conținând o implementare neconstantă a funcției merge.

```
class Merger m where
    merge :: m a -> m a -> m [a]
```

```
instance Merger Maybe where
    merge (Just x) (Just y) = Just [x, y]
    merge _        _        = Nothing
```

Supraîncărcați în Haskell operatorul de egalitate pentru funcții unare cu parametru numeric, astfel încât două funcții să fie considerate egale dacă valorile lor coincid în cel puțin 10 puncte din intervalul $1, \ldots, 100$.

Soluție:

```haskell
instance (Num a, Enum a, Eq b) => Eq (a -> b) where
    f == g = length (filter id [f x == g x | x <- [1..100]]) >= 10
```

REZOLUȚIE

Avem premisele, date în formă clauzală:

{ ¬P(x, y, z), ¬Q(z, x), R(x, y) }
{ P(Ion, Test, George) }
{ Q(George, Ion) }

Demonstrați *folosind rezoluția* că este adevărat R(Ion, Test).

Sunt 3 pași de rezoluție, cu P, Q, și R din prima clauză.

De exemplu:

```
¬R(Ion, Test)          // concluzia negată
¬P(x, y, z) V ¬Q(z, x) V R(x, y)
------------------------------ x <- Ion, y <- Test
¬P(Ion, Test, z) V ¬Q(z, Ion)
Q(George, Ion)
------------------------------ z <- George
¬P(Ion, Test, George)
P(Ion, Test, George)
------------------------
clauza vidă
```

Avem premisele, date în formă clauzală:

{ ¬P(x, y), ¬Q(x, y) }
{ ¬R(x, y), Q(x, y) }
{ P(x, y), R(x, y) }
{ P(Armand, Cadou) }
{ ¬P(Claudia, Cadou) }

Demonstrați *folosind rezoluția* că este adevărat Q(Claudia, Cadou).

Sunt 3 pași necesari, pentru
¬Q(Claudia, Cadou)
¬R(x, y), Q(x, y)

```
P(x, y), R(x, y)
¬P(Claudia, Cadou)

De exemplu:

¬Q(Claudia, Cadou)     // concluzia negată
¬R(x, y) V Q(x, y)
-------------------  x <- Claudia, y <- Cadou
¬R(Claudia, Cadou)
P(x, y) V R(x, y)
-------------------  x <- Claudia, y <- Cadou
P(Claudia, Cadou)
¬P(Claudia, Cadou)
-------------------
clauza vidă
```

Avem premisele, date în formă clauzală:

```
{ ¬P(x, y), ¬Q(z, t, x), R(t, z, y) }
{ Q(Ionuț, Aurel, Carioca) }
{ P(Carioca, Pachet) }
```

Demonstrați *folosind rezoluția* că este adevărat R(Aurel, Ionuț, Pachet).

Soluție:

Sunt 3 pași de rezoluție, cu P, Q, și R din prima clauză.

```
¬P(x, y) V ¬Q(z, t, x) V R(t, z, y)
¬R(Aurel, Ionuț, Pachet)          // concluzia negată
---------------------------- t <- Aurel, z <- Ionuț, y <- Pachet
¬P(x, Pachet) V ¬Q(Ionuț, Aurel, x)
Q(Ionuț, Aurel, Carioca)
---------------------------- x <- Carioca
¬P(Carioca, Pachet)
P(Carioca, Pachet)
-------------------
clauza vidă
```

```
Avem premisele, date în formă clauzală:

{ ¬P(x), ¬Q(y, x), ¬T(z), ¬R(z, x) }
{ P(Cadou) }
{ Q(12, Cadou) }
{ T(21) }

Demonstrați *folosind rezoluția* că este adevărat ¬R(21, Cadou).

Sunt 4 pași de rezoluție, cu P, Q, T și R din prima clauză.

De exemplu:

¬P(x) V ¬Q(y, x) V ¬T(z) V ¬R(z, x)
Q(12, Cadou)
--------------------------- x <- Cadou, y <- 12
¬P(Cadou) V ¬T(z) V ¬R(z, Cadou)
P(Cadou)
------------------------

  ¬T(z) V ¬R(z, Cadou)
  T(21)
  ---------------------- z <- 21
  ¬R(12, Cadou)
  R(21, Cadou)      // concluzia negată
  --------------
  Clauza vidă
```

PREDICATE DE ORDIN 1

7. Transformați propoziția „Nu tot ce zboară se mănâncă.” în logică cu predicate de ordinul întâi.
   Soluție:
   $\exists x.zboara(x) \wedge \neg se_mananca(x)$ – există și lucruri care zboară și nu se mănâncă
   sau
   $\neg(\forall x.zboara(x) \Rightarrow se_mananca(x))$ – nu este adevărat că orice care zboară automat se și mănâncă

7. Transformați propoziția „Nu mor caii când vor câinii.” în logică cu predicate de ordinul întâi, folosind predicatele `caii_mor(când)` și `câinii_vor(când)`.
   Soluție:
   $\exists t.cainii_vor(t) \wedge \neg caii_mor(t)$ – există și momente când câinii vor dar caii nu mor
   sau
   $\neg(\forall t.cainii_vor(t) \Rightarrow caii_mor(t))$ – nu este adevărat că oricând câinii vor automat caii mor

8. Traduceți în logica cu predicate de ordinul întâi propoziția: *Orice naș își are nașul.*
   Soluție:
   $\forall x \forall y.nas(x, y) \Rightarrow \exists z.nas(z, x)$
   sau
   $\forall x.(\exists y.nas(x, y)) \Rightarrow \exists z.nas(z, x)$

PROLOG

Implementați predicatul `filtersorted(+LLIn, -LLOut)`, care primește în `LLIn` o listă de liste de numere și pune în `LLOut` doar acele liste de numere care sunt sortate crescător. De exemplu, `filtersorted([[2, 1], [1, 3], [1, 2, 3, 4], [1, 2, 4, 2]], X)` leagă `X` la `[[1, 3], [1, 2, 3, 4]]`. Explicați cum funcționează implementarea.

Soluție:

```
filtersorted([], []).
filtersorted([E|LLIn], [E|LLOut]) :- sort(E, E), !, filtersorted(LLIn, LLOut).
filtersorted([_|LLIn], LLOut) :- filtersorted(LLIn, LLOut).
```

```
is_sorted([]).
is_sorted([_]).
is_sorted([X, Y|Rest]) :- X =< Y, is_sorted([Y|Rest]).

filtersorted([], []).
filtersorted([X|Rest], [X|RestAns]) :- is_sorted(X), filtersorted(Rest, RestAns).
filtersorted([_|Rest], Ans) :- filtersorted(Rest, Ans).
```

Implementați în Prolog predicatul identify(+ListsIn, -ListsOut) care pentru o listă de liste numerice, produce o listă de perechi, fiecare pereche (N, L) corespunzând unei liste L din ListsIn, iar numărul N fiind:
- maximul listei, dacă L este sortată crescător;
- primul element din L, altfel.

```
is_sorted([]).
is_sorted([_]).
is_sorted([X, Y|Rest]) :- X =< Y, is_sorted([Y|Rest]).


getMAX([X],X).
getMAX([X|L], RestMax) :- getMAX(L, RestMax), RestMax > X.
getMAX([X|L], X) :- getMAX(L, RestMax), RestMax =< X.

identify([], []).
identify([L|Rest], [(Max,L)|Ans]) :- is_sorted(L), getMAX(L, Max), identify(Rest,Ans).
identify([[First|L]|Rest], [(First,[First|L])|Ans]) :- \+ is_sorted([First|L]), identify(Rest,Ans).
```

PROLOG - FINDALL

Folosiți unul sau mai multe dintre predicatele `findall`, `forall`, `bagof`, `setof` pentru a implementa predicatul `med(+L, -Med)`, care găsește elementul median al listei, cu proprietatea că numărul de elemente mai mari decât `Med` este diferit cu cel mult 1 de numărul de elemente mai mici decât `Med`). Nu utilizați recursivitate explicită. Explicați cum funcționează soluția.

Soluție:

```
med(L, Med) :- member(Med, L), findall(X, (member(X, L), X < Med), L1),
               findall(X, (member(X, L), X > Med), L2), length(L1, LL1),
               length(L2, LL2), abs(LL1-LL2) =< 1.
```

```
Implementați folosind metapredicate (forall, findall, bagof, setof),
și fără recursivitate explicită, predicatul counts(+List, -ListPlus),
care primește în List o listă de liste de elemente oarecare, și leagă
ListPlus o listă de liste, fiecare element LP din ListPlus fiind
bazat pe o listă L din List: primul element este numărul elementelor
*diferite* din L, iar restul elementelor sunt elementele din L.

Exemplu:

counts([[a], [1, 2, 3, 4, 3], [x, y, z], [d, 1, d, 1], [e,
1,1,1,1,1,1]], X) leagă X la
[[1,a],[4,1,2,3,4,3],[3,x,y,z],[2,d,1,d,1],[2,e,1,1,1,1,1,1]].

Sol:

counts(List, ListPlus) :-
      findall([Len|L],
            (
                  member(L, List),
                  setof(E, member(E, L), Es),
                  length(Es, Len)
            ),
            ListPlus).

Barem:
Numărare elemente unice: 3p
Aplicare mecanism numărare elemente unice pe fiecare listă: 4p
Asamblare corectă: 3p
```

PROBLEMĂ

```haskell
type Text = String
-- pereche între un cuvânt și o linie din text
type Pair = (String, Int)
-- pereche între un cuvânt și lista tuturor liniilor pe care acesta apare
type Index = [(String, [Int])]
```

- (10p) Un text constă într-un string de cuvinte care pot fi separate prin spații sau enter (caracterul $\backslash n$). De asemenea, în text pot apărea semne de punctuație precum virgulă, punct, semnul întrebării sau semnul exclamării (după care apare minim un spațiu sau enter).

  Să se implementeze funcția `textToLines` care primește un text și întoarce o listă de perechi de forma: `(linie, text_aflat_pe acea_linie_procesat)`.

  Un text procesat reprezintă textul din care s-au eliminat semnele de punctuație iar literele mari au fost transformate în litere mici.

  ```haskell
  {-
  Utile:
  -- împarte un text în linii, parsând după '\n'
  lines :: String -> [String]
  -- transformă o literă mare în literă mică
  toLower :: Char -> Char

  ex:
  *Main> textToLines "A venit,\na venit\ntoamna!"
  [(1,"a venit"),(2,"a venit"),(3,"toamna")]
  -}

  textToLines :: Text -> [(Int, Text)]
  textToLines = undefined
  ```

- (10p) Să se implementeze funcția `insPair` care inserează o pereche `(cuvânt, linie)` într-un index, fără a crea linii duplicate.

```haskell
{-
ex:
*Main> insPair ("toamna",2) [("a", [1,2])]
[("toamna",[2]),("a",[1,2])]
*Main> insPair ("toamna",2) [("a", [1,2]), ("toamna", [1])]
[("toamna",[2,1]),("a",[1,2])]
*Main> insPair ("toamna",2) [("a", [1,2]), ("toamna", [1,2])]
[("a",[1,2]),("toamna",[1,2])] -- nu s-a mai adăugat încă o dată linia
↪  2
-}

insPair :: Pair -> Index -> Index
insPair = undefined
```

- (10p) Să se implementeze funcțiile `allPairs` și `textToIndex`. `allPairs` primește un text și creează toate perechile `(cuvânt, linie)` pentru acel text. `textToIndex` primește un text și creează indexul (sortat alfabetic după cuvânt) pentru acel text.

```haskell
{-
Utile: words :: String -> [String]        -- împarte un text în
↪  cuvinte, parsând după spații
Obs: tuplurile sunt ordonabile în Haskell (după primul element din
↪  tuplu, apoi al doilea, etc).

ex:
*Main> allPairs "A venit,\na venit\ntoamna!"
[("a",1),("venit",1),("a",2),("venit",2),("toamna",3)]
*Main> textToIndex "A venit,\na venit\ntoamna!"
[("a",[1,2]),("toamna",[3]),("venit",[1,2])]
-}
allPairs :: Text -> [Pair]
allPairs = undefined

textToIndex :: Text -> Index
textToIndex = undefined
```

Soluție:

```haskell
-1.
textToLines :: Text -> [(Int, Text)]
textToLines = zip [1 ..] . lines . map toLower . filter (`notElem` ",.?!")

--2.
insPair :: Pair -> Index -> Index
insPair (word, line) index = case lookup word index of
  Nothing -> (word, [line]) : index
  Just lines -> if elem line lines then index
                                   else (word, line : lines) : filter ((/=
                                   ↪  word) . fst) index

--3.
allPairs :: Text -> [Pair]
allPairs txt = [ (word, line) | (line, text) <- textToLines txt, word <-
↪  words text ]

textToIndex :: Text -> Index
textToIndex = sort . foldr insPair [] . allPairs
```

10. În acest exercițiu urmărim crearea și exploatarea unei tabele de frecvență pentru caracterele care apar în cuvinte diferite dintr-un text. O tabelă de frecvență este o colecție de asocieri între un caracter și frecvența sa de apariție.

    Utilizați următorul schelet:

```
import Data.List
import Data.Char
import Data.Maybe
```

- (10p) Să se creeze aliasul de tip `FreqTable`, pentru tabele de frecvență.

  Să se implementeze funcția `rareChars`, care primește o tabelă de frecvență și un număr n și întoarce lista caracterelor din tabelă care au frecvența mai mică decât n.

  Exemplu:

```
*Main> rareChars [('a',3), ('b',1), ('c',2), ('d',1)] 2
"bd"
```

```
type FreqTable = Int -- aici trebuie modificat în altceva decât Int

rareChars :: FreqTable -> Int -> [Char]
rareChars = undefined
```

- (10p) Să se implementeze funcția `insChar`, care inserează un caracter într-o tabelă de frecvență (dacă el exista deja în tabelă, atunci frecvența este actualizată).

  Exemplu:

```
*Main> insChar 'e' [('a',3), ('b',1), ('c',2), ('d',1)]
[('e',1),('a',3),('b',1),('c',2),('d',1)]
*Main> insChar 'c' [('a',3), ('b',1), ('c',2), ('d',1)]
[('c',3),('a',3),('b',1),('d',1)] -- s-a actualizat frecvența lui c
```

```
insChar :: Char -> FreqTable -> FreqTable
insChar = undefined
```

- (10p) Să se implementeze funcțiile `textToStr` și `textToTable`.

  `textToStr` primește un text și creează un string în care fiecare cuvânt din text apare o singură dată, și sunt eliminate spațiile albe.

  `textToTable` primește un text și creează o tabelă de frecvență pentru cuvintele distincte din acesta (folosind rezultatul lui `texToStr`).

  Util:

```
words :: String -> [String] -- împarte un text în cuvinte, parsând după
↪  spații
```

  Exemplu:

```
*Main> textToStr "copacii albi copacii negri"
"copaciialbinegri"
*Main> textToTable "copacii albi copacii negri"
[('c',2),('o',1),('p',1),('a',2),('i',4),('l',1),('b',1),('n',1),('e',1),('g',1),('r',1)]
```

```haskell
textToStr :: String -> String
textToStr = undefined

textToTable :: String -> FreqTable
textToTable = undefined
```

Soluție:

```haskell
--1.
type FreqTable = [(Char, Int)]

rareChars :: FreqTable -> Int -> [Char]
rareChars table n = [ c | (c, f) <- table, f < n ]

--2.
insChar :: Char -> FreqTable -> FreqTable
insChar c table = case lookup c table of -- nu e nevoie de lookup, merge și
↪  filter simplu sau recursivitate
   Nothing -> (c, 1) : table
   Just f -> (c, f+1) : filter ((/= c) . fst) table

--3.
textToStr :: String -> String
textToStr = concat . nub . words -- duplicatele se pot elimina și cu un fold
↪  simplu (ca la lab3)

textToTable :: String -> FreqTable
textToTable = foldr insChar [] . textToStr
```

textToTable text = foldr (\x acc -> insChar x acc) [ ] (textToStr text)

PROBLEME ++

5. (a) Câți pași de concatenare sunt realizați pentru evaluarea expresiei Racket `(car (append '(1 2) '(3 4)))` ?
   (b) Dar pentru expresia Haskell `head $ [1, 2] ++ [3, 4]` ?

Soluție:

(a) Se concatenează întregime listele, deci doi pași.
(b) Este suficient un singur pas pentru ca `head` să întoarcă primul element.